

Name: A Harika

Hallticket No:2303A51113

Batch:26

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING																																					
Program Name: B. Tech		Assignment Type: Lab	Academic Year: 2025-2026																																				
Course Coordinator Name		Dr. Rishabh Mittal																																					
Instructor(s)Name		<table border="1"> <tr><td>Mr. S Naresh Kumar</td><td></td></tr> <tr><td>Ms. B. Swathi</td><td></td></tr> <tr><td>Dr. Sasanko Shekhar Gantayat</td><td></td></tr> <tr><td>Mr. Md Sallauddin</td><td></td></tr> <tr><td>Dr. Mathivanan</td><td></td></tr> <tr><td>Mr. Y Srikanth</td><td></td></tr> <tr><td>Ms. N Shilpa</td><td></td></tr> <tr><td>Dr. Rishabh Mittal (Coordinator)</td><td></td></tr> <tr><td>Dr. R. Prashant Kumar</td><td></td></tr> <tr><td>Mr. Ankushavali MD</td><td></td></tr> <tr><td>Mr. B Viswanath</td><td></td></tr> <tr><td>Ms. Sujitha Reddy</td><td></td></tr> <tr><td>Ms. A. Anitha</td><td></td></tr> <tr><td>Ms. M.Madhuri</td><td></td></tr> <tr><td>Ms. Katherashala Swetha</td><td></td></tr> <tr><td>Ms. Velpula sumalatha</td><td></td></tr> <tr><td>Mr. Bingi Raju</td><td></td></tr> <tr><td>Mr. G. Kranthi</td><td></td></tr> </table>		Mr. S Naresh Kumar		Ms. B. Swathi		Dr. Sasanko Shekhar Gantayat		Mr. Md Sallauddin		Dr. Mathivanan		Mr. Y Srikanth		Ms. N Shilpa		Dr. Rishabh Mittal (Coordinator)		Dr. R. Prashant Kumar		Mr. Ankushavali MD		Mr. B Viswanath		Ms. Sujitha Reddy		Ms. A. Anitha		Ms. M.Madhuri		Ms. Katherashala Swetha		Ms. Velpula sumalatha		Mr. Bingi Raju		Mr. G. Kranthi	
Mr. S Naresh Kumar																																							
Ms. B. Swathi																																							
Dr. Sasanko Shekhar Gantayat																																							
Mr. Md Sallauddin																																							
Dr. Mathivanan																																							
Mr. Y Srikanth																																							
Ms. N Shilpa																																							
Dr. Rishabh Mittal (Coordinator)																																							
Dr. R. Prashant Kumar																																							
Mr. Ankushavali MD																																							
Mr. B Viswanath																																							
Ms. Sujitha Reddy																																							
Ms. A. Anitha																																							
Ms. M.Madhuri																																							
Ms. Katherashala Swetha																																							
Ms. Velpula sumalatha																																							
Mr. Bingi Raju																																							
Mr. G. Kranthi																																							
Course Code	23CS002PC304	Course Title	AI Assisted Coding																																				
Year/Sem	III/I	Regulation	R23																																				
Date and Day of Assignment	Week 9 - Thursday	Time(s)	23CSBTB01 To 23CSBTB52																																				
Duration	2 Hours	Applicable to Batches	All Batches																																				
AssignmentNumber: 18.4 (Present assignment number)/24(Total number of assignments)																																							
Q.No.	Question		ExpectedTime to complete																																				
1	Lab 18: API Integration: Connecting to External Services with Error Handling		Week 9																																				

	Lab Objectives • Learn how to integrate Python programs with external REST APIs. • Understand API request/response handling using requests or similar libraries. • Implement proper error handling for failed API calls (timeouts, invalid responses, rate limits). • Practice extracting and displaying meaningful information from API responses. Learning Outcome After completing this assignment, students will be able to: • Integrate external REST APIs using AI-assisted Python scripting. • Implement structured error handling for API failures and invalid inputs. • Safely parse and validate JSON responses from external services. • Develop resilient applications capable of handling real-world API limitations.	
	Task 1 – Movie Database API Integration Scenario You are building a movie recommendation tool that fetches movie details from a public Movie Database API. Task Use AI to generate a Python script that connects to a Movie API and retrieves movie information by title. Instructions • Use <code>requests</code> library to fetch movie details. • Accept movie title as user input. • Display: ◦ Title ◦ Release year ◦ Rating ◦ Plot summary • Implement error handling for: ◦ Invalid movie name ◦ API key errors ◦ Network timeout	

- Empty API response

Expected Output

- A working Python script that fetches movie details.
- Graceful error messages instead of crashes.
- Clean, readable output format.

```
C:\Users> prade > OneDrive > Pradeep > python language > classnodepy > fetch_movie_details

1 import requests
2 import sys
3 from typing import Optional, Dict
4
5 # OMDB API endpoint
6 OMDB_API_URL = "http://www.omdbapi.com/"
7
8 # Get your free API key from: https://www.omdbapi.com/apkey.aspx
9 API_KEY = "YOUR_API_KEY_HERE"
10
11 # Timeout for API requests (in seconds)
12 REQUEST_TIMEOUT = 5
13
14 def fetch_movie_details(title: str) -> Optional[Dict]:
15     """
16     Fetch movie details from OMDB API by title.
17
18     Args:
19         title: Movie title to search for
20
21     Returns:
22         Dictionary containing movie information or None if error occurs
23
24     Raises:
25         Handles exceptions internally and returns None
26     """
27     # Validate input
28     if not title or not title.strip():
29         print(f"Error: Movie title cannot be empty.")
30         return None
31
32     # Prepare API request parameters
33     params = {
34         "apikey": API_KEY,
35         "t": title.strip(),
36         "type": "movie"
37     }
38
39     # Make API request with timeout
40     response = requests.get(OMDB_API_URL, params=params, timeout=REQUEST_TIMEOUT)
41
42     # Check HTTP status code
43     if response.status_code != 200:
44         print(f"Error: API returned status code {response.status_code}")
45         return None
46
47     # Parse JSON response
48     data = response.json()
49
50     # Check for API key error
51     if "Error" in data:
52         error_message = data["Error"]
53         if "Invalid api key" in error_message.lower():
54             print(f"API Key Error: Please set a valid API key.")
55             print(f"Get a free key at: https://www.omdbapi.com/apkey.aspx")
56         elif "movie not found" in error_message.lower():
57             print(f"Error: Not Found: No movie found with title '{title}'.")
58         else:
59             print(f"Error: {error_message}")
60             return None
61
62     # Check for empty response
63     if not data or data == {}:
64         print(f"Error: Empty response from API")
65         return None
66
67     return data
68
69 except requests.exceptions.Timeout:
70     print(f"Error: Request timed out (timeout: {REQUEST_TIMEOUT}s).")
71     print(f"Check your internet connection and try again.")
72     return None
73
74 except requests.exceptions.ConnectionError:
75     print(f"Error: Unable to connect to the API server.")
76     print(f"Please check your internet connection.")
77     return None
78
79 except requests.exceptions.RequestException as e:
80     print(f"Error: Network error occurred - {str(e)}")
81     return None
82
83 except ValueError:
84     print(f"Error: Invalid response format from API")
85     return None
86
87 def display_movie_info(movie_data: Dict) -> None:
88     """
89     Display movie information in a formatted way.
90
91     Args:
92         movie_data: Dictionary containing movie information
93     """
94
95     # Extract movie details with fallbacks
96     title = movie_data.get("Title", "N/A")
97     year = movie_data.get("Year", "N/A")
98     rating = movie_data.get("imdbRating", "N/A")
99     plot = movie_data.get("Plot", "N/A")
100     director = movie_data.get("Director", "N/A")
101     runtime = movie_data.get("Runtime", "N/A")
102     genre = movie_data.get("Genre", "N/A")
103
104     # Display formatted output
105     print("\n" + "="*60)
106     print(f"Movie Details")
107     print(f"Title: {title}")
108     print(f"Release Year: {year}")
109     print(f"IMDb Rating: {rating}/10")
110     print(f"Runtime: {runtime}")
111     print(f"Genre: {genre}")
112     print(f"Director: {director}")
113     print(f"Plot: {plot}")
114     print(f"Summary: {plot}")
115     print("\n" + "="*60 + "\n")
```

```

123
124 def main():
125     """Main function to run the movie search tool."""
126
127     print("\n" + "="*60)
128     print("  MOVIE DATABASE API INTEGRATION")
129     print("="*60)
130
131     # Check if API key is set
132     if API_KEY == "YOUR_API_KEY_HERE":
133         print("\n⚠ WARNING: API key not configured!")
134         print("Please follow these steps:")
135         print("1. Visit: https://www.omdbapi.com/apikey.aspx")
136         print("2. Sign up for a free API key")
137         print("3. Replace 'YOUR_API_KEY_HERE' in this script with your key")
138         print("\nOrganizations can get a dedicated API key for commercial use.")
139         return
140
141     # Get movie title from user
142     try:
143         print("\nEnter 'quit' or 'exit' to close the program.\n")
144
145         while True:
146             movie_title = input("👉 Enter movie title to search: ").strip()
147
148             # Check for exit command
149             if movie_title.lower() in ['quit', 'exit', 'q']:
150                 print("\n👋 Thank you for using Movie Database API!")
151                 break
152
153             # Fetch movie details
154             movie_data = fetch_movie_details(movie_title)
155
156             # Display results if found
157             if movie_data:
158                 display_movie_info(movie_data)
159
160         except KeyboardInterrupt:
161             print("\n\n⏹ Program interrupted by user.")
162             sys.exit(0)
163         except Exception as e:
164             print(f"\n❌ Unexpected error: {str(e)}")
165             sys.exit(1)
166
167     if __name__ == "__main__":
168         main()
169
170

```

C:\Users\prade>C:\Users\prade\AppData\Local\Microsoft\WindowsApps\python3.12.exe "c:/Users/prade/OneDrive/Pradeep/python language/classnode.py"

```

=====
MOVIE DATABASE API INTEGRATION
=====

```

```

⚠ WARNING: API key not configured!
Please follow these steps:
1. Visit: https://www.omdbapi.com/apikey.aspx

```

C:\Users\prade>C:\Users\prade\AppData\Local\Microsoft\WindowsApps\python3.12.exe "c:/Users/prade/OneDrive/Pradeep/python language/classnode.py"

```

=====
MOVIE DATABASE API INTEGRATION
=====

```

```

⚠ WARNING: API key not configured!
Please follow these steps:
1. Visit: https://www.omdbapi.com/apikey.aspx

```

```

=====
MOVIE DATABASE API INTEGRATION
=====

```

```

⚠ WARNING: API key not configured!
Please follow these steps:
1. Visit: https://www.omdbapi.com/apikey.aspx

```

```

⚠ WARNING: API key not configured!
Please follow these steps:
1. Visit: https://www.omdbapi.com/apikey.aspx
2. Sign up for a free API key
Please follow these steps:
1. Visit: https://www.omdbapi.com/apikey.aspx
2. Sign up for a free API key
3. Replace 'YOUR_API_KEY_HERE' in this script with your key
3. Replace 'YOUR_API_KEY_HERE' in this script with your key

```

Organizations can get a dedicated API key for commercial use.

```

C:\Users\prade>
Organizations can get a dedicated API key for commercial use.

```

```

Organizations can get a dedicated API key for commercial use.
Organizations can get a dedicated API key for commercial use.
Organizations can get a dedicated API key for commercial use.

```

Task 2 – Public Transport Schedule API

Scenario

	A city transport app needs to fetch bus/train arrival timings from a transport API. Task Use AI assistance to integrate a transport schedule API. Instructions • Fetch arrival times for a given station/stop ID. • Validate user input. • Handle errors such as: ◦ Invalid stop ID ◦ Server unavailable ◦ Timeout errors • Display next 3 arrival times in formatted output. • Implement retry logic (minimum 1 retry attempt). Expected Output • Python script showing upcoming arrivals. • Retry mechanism working properly. • Proper exception handling using try-except blocks.	
--	---	--

```

C:\Users\prade > OneDrive > Pradeep > python language > classnodepy > ...
1 import requests
2 import time
3 from typing import Optional, List, Dict
4
5
6 MAX_RETRIES = 2
7 TIMEOUT = 10
8
9 # Mock data (replace with real API in production)
10 SCHEDULE_DB = {
11     "101": [{"time": "14:30", "dest": "Central", "line": "Bus 101"},
12             {"time": "14:45", "dest": "Central", "line": "Bus 101"},
13             {"time": "15:00", "dest": "Central", "line": "Bus 101"}],
14     "205": [{"time": "14:35", "dest": "Airport", "line": "Train 205"},
15             {"time": "14:50", "dest": "Airport", "line": "Train 205"},
16             {"time": "15:05", "dest": "Airport", "line": "Train 205"}],
17     "312": [{"time": "14:25", "dest": "Harbor", "line": "Bus 312"},
18             {"time": "14:40", "dest": "Harbor", "line": "Bus 312"},
19             {"time": "15:10", "dest": "Harbor", "line": "Bus 312"}],
20 }
21
22 def validate_stop_id(stop_id: str) -> bool:
23     """Validate stop ID format and existence."""
24     if not stop_id or not stop_id.strip():
25         print("❌ Error: Stop ID cannot be empty")
26         return False
27
28     stop_id = stop_id.strip()
29     if not stop_id.isalnum():
30         print("❌ Error: Invalid characters in '{stop_id}'")
31         return False
32
33     if stop_id not in SCHEDULE_DB:
34         print("❌ Error: Stop '{stop_id}' not found")
35         print(f"Available: {', '.join(SCHEDULE_DB.keys())}")
36         return False
37
38     return True
39
40 def fetch_schedule_with_retry(stop_id: str) -> Optional[List[Dict]]:
41     """Fetch schedule with retry logic and error handling."""
42     for attempt in range(MAX_RETRIES):
43         try:
44             print(f"🔄 Fetching schedule (Attempt {attempt + 1}/{MAX_RETRIES})...")
45
46             # Simulate API call
47             schedule = SCHEDULE_DB.get(stop_id)
48
49             if schedule:
50                 print("✅ Schedule fetched successfully")
51                 return schedule
52
53             raise Exception("No data found")
54
55         except requests.exceptions.Timeout:
56             print(f"⌚ Timeout error")
57             if attempt < MAX_RETRIES - 1:
58                 wait = 2 ** attempt
59                 print(f"🔄 Retrying in {wait}s...")
60                 time.sleep(wait)
61             else:
62                 print("❌ Failed: Server not responding")
63
64 def fetch_schedule_with_retry(stop_id: str) -> Optional[List[Dict]]:
65     except requests.exceptions.ConnectionError:
66         print(f"🔌 Connection error")
67         if attempt < MAX_RETRIES - 1:
68             wait = 2 ** attempt
69             print(f"🔄 Retrying in {wait}s...")
70             time.sleep(wait)
71         else:
72             print("❌ Failed: Cannot connect to server")
73
74     except Exception as e:
75         print(f"❌ Error: {str(e)}")
76         if attempt < MAX_RETRIES - 1:
77             print(f"🔄 Retrying...")
78             time.sleep(1)
79         else:
80             print("❌ Failed: Max retries exceeded")
81
82     return None
83
84 def display_arrivals(stop_id: str, schedule: List[Dict]) -> None:
85     """Display next 3 arrivals in formatted output."""
86     print(f"\n{'*' * 55}")
87     print(f"📍 Stop ID: {stop_id}")
88     print(f"{'*' * 55}")
89     print(f"🕒 {'Arrival Time':<15} 📍 {'Destination':<20} 🚏 {'Line':<15}")
90     print(f"{'*' * 55}")
91
92     for i, arrival in enumerate(schedule[:3], 1):
93         print(f"🕒 {i:<3} 🕒 {arrival['time']:<15} 📍 {arrival['dest']:<20} 🚏 {arrival['line']:<15}")
94
95     print(f"{'*' * 55}\n")
96
97 def get_user_input() -> Optional[str]:
98     """Get user input with quit option."""
99     try:
100         stop_id = input("\n📍 Enter Stop ID (or 'quit'): ").strip()
101         return None if stop_id.lower() == 'quit' else stop_id
102     except KeyboardInterrupt:
103         print("\n👋 Exiting...")
104         return None
105
106 def main():
107     """Main application loop."""
108     print("\n" + "=" * 53 + "\n")
109     print("\n" + "PUBLIC TRANSPORT SCHEDULE API".center(53) + "\n")
110     print("\n" + "-" * 53 + "\n")
111
112     while True:
113         stop_id = get_user_input()
114         if stop_id is None:
115             break
116
117         if not validate_stop_id(stop_id):
118             continue
119
120         schedule = fetch_schedule_with_retry(stop_id.strip())
121
122         if schedule:
123             display_arrivals(stop_id.strip(), schedule)
124         else:
125             print(f"❌ Could not retrieve schedule for '{stop_id}'")
126
127 if __name__ == "__main__":
128     main()

```

C:\Users\pradeeC:\Users\prade/AppData/Local/Microsoft/WindowsApps/python3.12.exe "c:/Users/prade/OneDrive/Pradeep/python language/classnode.py"

PUBLIC TRANSPORT SCHEDULE API

Enter Stop ID (or 'quit'): 2303A1522

Error: Stop '2303A1522' not found
Available: 101, 205, 312

Enter Stop ID (or 'quit'): C:/Users/prade/AppData/Local/Microsoft/WindowsApps/python3.12.exe "c:/Users/prade/OneDrive/Pradeep/python language/classnode.py"
Error: Invalid characters in 'C:/Users/prade/AppData/Local/Microsoft/WindowsApps/python3.12.exe "c:/Users/prade/OneDrive/Pradeep/python language/classnode.py"

Enter Stop ID (or 'quit'): 205

Fetching schedule (Attempt 1/2)...
Schedule fetched successfully

Stop ID: 205

#	Arrival Time	Destination	Line
1	14:35	Airport	Train 205
2	14:50	Airport	Train 205
3	15:05	Airport	Train 205

Enter Stop ID (or 'quit'): 101

Fetching schedule (Attempt 1/2)...
Schedule fetched successfully

Stop ID: 101

#	Arrival Time	Destination	Line
1	14:30	Central	Bus 101
2	14:45	Central	Bus 101
3	15:00	Central	Bus 101

Enter Stop ID (or 'quit'): 312

Fetching schedule (Attempt 1/2)...
Schedule fetched successfully

Stop ID: 312

#	Arrival Time	Destination	Line
1	14:25	Harbor	Bus 312
2	14:40	Harbor	Bus 312
3	15:10	Harbor	Bus 312
3	15:05	Airport	Train 205

Enter Stop ID (or 'quit'): 101

Fetching schedule (Attempt 1/2)...
Schedule fetched successfully

Stop ID: 101

#	Arrival Time	Destination	Line
1	14:30	Central	Bus 101
2	14:45	Central	Bus 101
3	15:00	Central	Bus 101

Enter Stop ID (or 'quit'): 312

Fetching schedule (Attempt 1/2)...
Schedule fetched successfully

Stop ID: 312

#	Arrival Time	Destination	Line
1	14:25	Harbor	Bus 312
2	14:40	Harbor	Bus 312
3	15:10	Harbor	Bus 312

Fetching schedule (Attempt 1/2)...
Schedule fetched successfully

Stop ID: 101

#	Arrival Time	Destination	Line
1	14:30	Central	Bus 101
2	14:45	Central	Bus 101
3	15:00	Central	Bus 101

Enter Stop ID (or 'quit'): 312

Fetching schedule (Attempt 1/2)...
Schedule fetched successfully

Stop ID: 312

#	Arrival Time	Destination	Line
1	14:25	Harbor	Bus 312
2	14:40	Harbor	Bus 312
3	15:10	Harbor	Bus 312
3	15:00	Central	Bus 101

```

Enter Stop ID (or 'quit'): 312

Fetching schedule (Attempt 1/2)...
Schedule fetched successfully

=====
Stop ID: 312
=====
#   Arrival Time   Destination   Line
-----
1   14:25          Harbor       Bus 312
2   14:40          Harbor       Bus 312
3   15:10          Harbor       Bus 312
=====

Schedule fetched successfully

=====
Stop ID: 312
=====
#   Arrival Time   Destination   Line
-----
1   14:25          Harbor       Bus 312
2   14:40          Harbor       Bus 312
3   15:10          Harbor       Bus 312
=====

#   Arrival Time   Destination   Line
-----
1   14:25          Harbor       Bus 312
2   14:40          Harbor       Bus 312
3   15:10          Harbor       Bus 312
=====

3   15:10          Harbor       Bus 312
=====

Enter Stop ID (or 'quit'): 

```

Task 3 – Stock Market Data API

Scenario

You are developing a finance dashboard that retrieves live stock prices.

Task

Use AI to generate a Python script that fetches stock data from a financial API.

Instructions

- Accept stock symbol as input.
- Retrieve:

	○ Current price ○ Day high ○ Day low ● Handle: ○ Invalid stock symbol ○ API rate limits ○ Missing fields in JSON response ● Validate and safely parse JSON response. Expected Output ● Script fetches and displays stock information. ● No unhandled exceptions. ● Clear error messages for invalid symbols or rate limits.	
--	--	--

```

C:\Users > prade > OneDrive > Pradeep > python language > classnode.py > validate_stock_symbol

2 import requests
3 import time
4 from typing import Optional, Dict
5
6 API_KEY = "demo"
7 BASE_URL = "https://www.alphavantage.co/query"
8 TIMEOUT = 10
9 api_calls = []
10 MAX_CALLS = 5
11
12 class StockAPIException(Exception):
13     pass
14
15 def validate_stock_symbol(symbol: str) -> bool:
16     """Validate stock symbol format."""
17     if not symbol or not isinstance(symbol, str):
18         print("❌ Error: Stock symbol must be non-empty string")
19         return False
20
21     symbol = symbol.strip().upper()
22     if not (1 <= len(symbol) <= 5) or not symbol.isalpha():
23         print("❌ Error: Invalid symbol format (1-5 letters only)")
24         return False
25     return True
26
27 def check_rate_limit() -> bool:
28     """Check API rate limit (5 requests per minute)."""
29     global api_calls
30     current = time.time()
31     api_calls = [t for t in api_calls if current - t < 60]
32
33     if len(api_calls) >= MAX_CALLS:
34         print("❌ Rate Limit: Max 5 requests/minute exceeded")
35         print(f"⚠️ Retry in {60 - (current - api_calls[0]):.1f} seconds")
36         return False
37     api_calls.append(current)
38     return True
39
40 def fetch_stock_data(symbol: str) -> Optional[Dict]:
41     """Fetch stock data from API with error handling."""
42     try:
43         if not check_rate_limit():
44             raise StockAPIException("Rate limit exceeded")
45
46         print(f"🔄 Fetching {symbol}...")
47
48         params = {"function": "GLOBAL_QUOTE", "symbol": symbol.upper(), "apikey": API_KEY}
49         response = requests.get(BASE_URL, params=params, timeout=TIMEOUT)
50         response.raise_for_status()
51
52         data = response.json()
53
54         # Check for API errors
55         if "Error Message" in data:
56             raise StockAPIException(f"Invalid symbol '{symbol}' - not found")
57         if "Note" in data:
58             raise StockAPIException("API rate limit reached")
59
60         return data
61     except requests.exceptions.Timeout:
62         raise StockAPIException("❌ Timeout: API server not responding")
63
64
C:\Users > prade > OneDrive > Pradeep > python language > classnode.py > validate_stock_symbol
41 def fetch_stock_data(symbol: str) -> Optional[Dict]:
42     except requests.exceptions.ConnectionError:
43         raise StockAPIException("❌ Connection Error: Cannot reach API server")
44     except requests.exceptions.RequestException as e:
45         raise StockAPIException(f"❌ API Error: {str(e)}")
46
47 def validate_and_parse(data: Dict) -> Optional[Dict]:
48     """Validate and parse JSON response safely."""
49     try:
50         if "Global Quote" not in data or not data["Global Quote"]:
51             print("❌ Error: Invalid API response structure")
52             return None
53
54         quote = data["Global Quote"]
55
56         # Required fields with fallback
57         stock_info = {
58             "symbol": quote.get("01. symbol", "N/A"),
59             "price": quote.get("05. price", "N/A"),
60             "high": quote.get("03. high", "N/A"),
61             "low": quote.get("04. low", "N/A"),
62             "timestamp": quote.get("07. latest trading day", "N/A")
63         }
64
65         if not stock_info["price"]:
66             print("❌ Error: Current price not available")
67             return None
68         return stock_info
69     except (KeyError, TypeError) as e:
70         print(f"❌ Parse Error: {str(e)}")
71         return None
72
73 def display_stock_info(info: Dict) -> None:
74     """Display formatted stock information."""
75     print(f"\n💡 STOCK INFORMATION")
76     print(f"{'='*50}")
77     print(f"Symbol: {info['symbol']}")
78     print(f"Current Price: ${info['price']}")
79     print(f"Day High: ${info['high']}")
80     print(f"Day Low: ${info['low']}")
81     print(f"Last Updated: {info['timestamp']}")
82     print(f"{'='*50}\n")
83
84 def get_user_input() -> Optional[str]:
85     """Get stock symbol from user."""
86     try:
87         symbol = input("\n Enter stock symbol (or 'quit'): ").strip()
88         return None if symbol.lower() == 'quit' else symbol
89     except KeyboardInterrupt:
90         print("\n 🛑 Exiting...")
91         return None
92
93 def main():
94     """Main application loop."""
95     print("\n 📈 *48 * 1")
96     print("|" + "STOCK MARKET DATA API".center(48) + "|")
97     print("|" + "-"*48 + "-"*)
98

```

```
index.html  # style.css  JS script.js  layout.html  # layoutstyle.css  cta.html  JS cl
C: > Users > prade > OneDrive > Pradeep > python language > classnode.py > validate_stock_symbol

119 def main():
120     while True:
121         symbol = get_user_input()
122         if symbol is None:
123             break
124         if not validate_stock_symbol(symbol):
125             continue
126         try:
127             raw_data = fetch_stock_data(symbol)
128             if raw_data:
129                 stock_info = validate_and_parse(raw_data)
130                 if stock_info:
131                     display_stock_info(stock_info)
132             except StockAPIException as e:
133                 print(f"{str(e)}")
134             except Exception as e:
135                 print(f"✗ Unexpected Error: {str(e)}")
136         if __name__ == "__main__":
137             main()
138
139 Enter Stop ID (or 'quit'): C:\Users\prade\AppData\Local\Microsoft\WindowsApps\python3.12.exe "C:\Users\prade/OneDrive/Pradeep/python language/classnode.py"
140 ✗ Error: Invalid characters in "C:\Users\prade\AppData\Local\Microsoft\WindowsApps\python3.12.exe "C:\Users\prade/OneDrive/Pradeep/python language/classnode.py"
141 ✗ Error: Invalid characters in "C:\Users\prade\AppData\Local\Microsoft\WindowsApps\python3.12.exe "C:\Users\prade/OneDrive/Pradeep/python language/classnode.py"
142 ✗ Error: Invalid characters in "C:\Users\prade\AppData\Local\Microsoft\WindowsApps\python3.12.exe "C:\Users\prade/OneDrive/Pradeep/python language/classnode.py"
143 Enter Stop ID (or 'quit'):
144 ✗ Error: Invalid characters in "C:\Users\prade\AppData\Local\Microsoft\WindowsApps\python3.12.exe "C:\Users\prade/OneDrive/Pradeep/python language/classnode.py"
145 ✗ Error: Invalid characters in "C:\Users\prade\AppData\Local\Microsoft\WindowsApps\python3.12.exe "C:\Users\prade/OneDrive/Pradeep/python language/classnode.py"
146 Enter Stop ID (or 'quit'): 301
147 Fetching schedule (Attempt 1/2)...
148 Schedule fetched successfully
149
150 Stop ID: 301
151 =====
152 # Arrival Time Destination Line
153 -----
154 1 14:30 Central Bus 301
155 2 14:45 Central Bus 301
156 3 15:00 Central Bus 301
157 =====
158 Enter Stop ID (or 'quit'): 312
159 Fetching schedule (Attempt 1/2)...
160 Schedule fetched successfully
161
162 Stop ID: 312
163 =====
164 # Arrival Time Destination Line
165 -----
166 1 14:25 Harbor Bus 312
167 2 14:40 Harbor Bus 312
168 3 15:10 Harbor Bus 312
169 =====
170 Enter Stop ID (or 'quit'): 205
171 Fetching schedule (Attempt 1/2)...
172 Schedule fetched successfully
173
174 Stop ID: 205
175 =====
176 # Arrival Time Destination Line
177 -----
178 1 14:35 Airport Train 205
179 2 14:50 Airport Train 205
180 3 15:05 Airport Train 205
181 =====
182 # Arrival Time Destination Line
183 -----
184 1 14:35 Airport Train 205
185 2 14:50 Airport Train 205
186 3 15:05 Airport Train 205
187 =====
```

Task 4 – Geolocation API (IP Address Lookup)

Scenario

A cybersecurity tool needs to detect the geographical location of a user based on IP address.

Task

Integrate a Geolocation API using AI assistance.

Instructions

- Accept IP address as input.
- Fetch:
 - Country
 - City
 - ISP
- Validate IP format before making API call.
- Handle:
 - Invalid IP format
 - API failure
 - JSON decoding errors

Expected Output

- A script that returns geolocation details.
- Input validation implemented.
- Robust error handling included.

```

C:\Users\prade > OneDrive > Pradeep > python language > classnode.py > ...
1 import requests
2 import re
3 import json
4 from typing import Optional, Dict
5
6 API_URL = "https://ipapi.co/ip/json/"
7 TIMEOUT = 10
8
9
10
11 def validate_ip_format(ip: str) -> bool:
12     """Validate IPv4 address format."""
13     if not ip or not isinstance(ip, str):
14         print("❌ Error: IP address must be a non-empty string")
15         return False
16
17     ip = ip.strip()
18     ipv4_pattern = r'^(:?25[0-5]|2[0-4][0-9]?|0[0-9]?|0-9)?(\.){3}(:?25[0-5]|2[0-4][0-9]?|0[0-9]?|0-9)?$'
19
20     if not re.match(ipv4_pattern, ip):
21         print("❌ Error: Invalid IP format '{ip}'")
22         print("Please enter a valid IPv4 address (e.g., 8.8.8.8)")
23         return False
24
25     return True
26
27
28 def fetch_geolocation(ip: str) -> Optional[Dict]:
29     """Fetch geolocation data from API with error handling."""
30     try:
31         print(f"📡 Fetching geolocation for {ip}...")
32
33         response = requests.get(API_URL.format(ip=ip), timeout=TIMEOUT)
34         response.raise_for_status()
35
36         data = response.json()
37         print("✅ Geolocation data retrieved successfully")
38         return data
39
40     except requests.exceptions.Timeout:
41         print("❌ Timeout Error: API request took too long")
42         return None
43
44     except requests.exceptions.ConnectionError:
45         print("❌ Connection Error: Cannot reach geolocation API")
46         return None
47
48     except requests.exceptions.HTTPError as e:
49         print(f"❌ HTTP Error: {response.status_code}")
50         return None
51
52     except requests.exceptions.RequestException as e:
53         print(f"❌ Request Error: {str(e)}")
54         return None
55
56     except json.JSONDecodeError:
57         print("❌ JSON Decoding Error: Invalid response from API")
58         return None
59
60
61 def validate_and_parse_response(data: Dict) -> Optional[Dict]:
62     """Validate and parse geolocation response."""
63     try:
64         if not isinstance(data, dict):
65             print("❌ Error: Invalid response format")
66             return None
67
68         if "error" in data and data.get("error"):

```

```

69         print(f"❌ API Error: {data.get('reason', 'Unknown error')}")
70         return None
71
72     geolocation = {
73         "ip": data.get("ip", "N/A"),
74         "country": data.get("country_name", "N/A"),
75         "city": data.get("city", "N/A"),
76         "isp": data.get("org", "N/A"),
77         "latitude": data.get("latitude", "N/A"),
78         "longitude": data.get("longitude", "N/A"),
79         "region": data.get("region", "N/A")
80     }
81
82     if geolocation["country"] == "N/A" or geolocation["city"] == "N/A":
83         print("⚠ Warning: Some geolocation fields are unavailable")
84
85     return geolocation
86
87 except (KeyError, TypeError) as e:
88     print(f"❌ Error: Failed to parse data - {str(e)}")
89     return None
90
91
92 def display_geolocation_info(geo_info: Dict) -> None:
93     """Display geolocation information in formatted output."""
94     print(f"\n{'='*55}")
95     print(f"🌐 GEOLOCATION INFORMATION")
96     print(f"{'='*55}")
97     print(f"IP Address:      {geo_info['ip']}")
98     print(f"Country:           {geo_info['country']}")
99     print(f"Region:            {geo_info['region']}")
100    print(f"City:              {geo_info['city']}")
101    print(f"ISP:               {geo_info['isp']}")
102    print(f"Latitude:          {geo_info['latitude']}")
103    print(f"Longitude:         {geo_info['longitude']}")
104    print(f"{'='*55}\n")
105
106
107 def get_user_input() -> Optional[str]:
108     """Get IP address from user input."""
109     try:
110         ip = input("\n📍 Enter IP address (or 'quit' to exit): ").strip()
111         return None if ip.lower() == 'quit' else ip
112     except KeyboardInterrupt:
113         print("\n\n⏹ Interrupted by user. Exiting...")
114         return None
115
116
117 def main():
118     """Main application loop."""
119     print("\n" + "="*53 + "\n")
120     print("\n" + "GEOLOCATION API (IP Address Lookup)".center(53) + "\n")
121     print("\n" + "="*53 + "\n")
122
123     while True:
124         ip = get_user_input()
125         if ip is None:
126             print("💡 Thank you for using Geolocation API!")
127             break
128
129         if not validate_ip_format(ip):
130             continue
131
132         raw_data = fetch_geolocation(ip.strip())
133

```

```

134     if raw_data is None:
135         print(f"❌ Could not retrieve data for '{ip}'")
136         continue
137
138     geo_info = validate_and_parse_response(raw_data)
139
140     if geo_info:
141         display_geolocation_info(geo_info)
142     else:
143         print(f"❌ Failed to parse geolocation data for '{ip}'")
144
145
146 if __name__ == "__main__":
147     main()

```

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  SPELL CHECKER
-----
# Arrival Time  Destination  Line
-----
1 14:30         Central    Bus 101
2 14:45         Central    Bus 101
3 15:00         Central    Bus 101
-----

Enter Stop ID (or 'quit'): 315
❌ Error: Stop '315' not found
Available: 101, 205, 312

Enter Stop ID (or 'quit'): 312
Fetching schedule (Attempt 1/2)...
Schedule fetched successfully

Stop ID: 312
-----
# Arrival Time  Destination  Line
-----
1 14:25         Harbor     Bus 312
2 14:40         Harbor     Bus 312
3 15:10         Harbor     Bus 312
-----

Enter Stop ID (or 'quit'): 205
Fetching schedule (Attempt 1/2)...
Schedule fetched successfully

Stop ID: 205
-----
# Arrival Time  Destination  Line
-----
1 14:35         Airport    Train 205
2 14:50         Airport    Train 205
3 15:05         Airport    Train 205
-----

```

Task 5 – Real-Time Application: Payment Gateway Simulation API

Scenario

You are building a test payment integration system that connects to a sandbox payment API.

Task

Use AI to generate a Python script that simulates payment processing.

Instructions

- Send POST request with:
 - Amount
 - Currency
 - Payment method
- Validate input values.
- Handle:
 - Invalid payment details
 - HTTP status errors (400, 401, 500)
 - Network interruptions

- Display transaction status clearly.
- Log failed transactions to a file.

Expected Output

- Script processes simulated payment.
- Errors handled gracefully.
- Failed requests logged properly.

```

1  #!/usr/bin/env python3
2  """
3  Simulate a payment transaction.
4  """
5  import sys
6  import json
7  import logging
8  import random
9  import time
10
11  # Configuration
12  URL = "https://api.example.com/v1/payments"
13  LOG_FILE = "payments_transactions.log"
14
15  # Logging setup
16  logging.basicConfig(
17      level=logging.INFO,
18      format='%(asctime)s - %(message)s',
19      handlers=[logging.FileHandler(LOG_FILE), logging.StreamHandler()]
20  )
21
22  def validate_amount(amount: str) -> bool:
23      """Validate amount"""
24      try:
25          amount_float = float(amount)
26          if amount_float < 0:
27              print(f"Error: Amount must be greater than 0")
28              return False
29          if amount_float > 1000000:
30              print(f"Error: Amount exceeds maximum limit")
31              return False
32          return True
33      except ValueError:
34          print(f"Error: Amount must be a valid number")
35          return False
36
37  def validate_currency(currency: str) -> bool:
38      """Validate currency code"""
39      valid_currencies = ["USD", "EUR", "GBP", "JPY", "AUD", "CAD"]
40      if not currency:
41          print(f"Error: Currency required")
42          return False
43      if currency not in valid_currencies:
44          print(f"Error: Invalid currency. Supported: {', '.join(valid_currencies)}")
45          return False
46      return True
47
48  def validate_payment_method(method: str) -> bool:
49      """Validate payment method"""
50      valid_methods = ["credit_card", "debit_card", "digital_wallet", "net_banking"]
51      if not method:
52          print(f"Error: Payment method required")
53          return False
54      if method not in valid_methods:
55          print(f"Error: Invalid payment method. Valid: {', '.join(valid_methods)}")
56          return False
57      return True
58
59  def process_payment(amount: float, currency: str, method: str) -> Optional[dict]:
60      """Process payment via various API"""
61      payload = {
62          "amount": amount,
63          "currency": currency,
64          "payment_method": method,
65          "timestamp": datetime.now().isoformat()
66      }
67
68      try:
69          # Simulating payment process
70          response = requests.post(URL, json=payload, timeout=10)
71
72

```

```

74     if response.status_code == 200:
75         print("✅ Payment processed successfully")
76         return response.json()
77     elif response.status_code == 400:
78         print("❌ Error 400: Invalid payment details")
79         logging.error(f"Bad Request - {payload} - {response.text}")
80         return None
81     elif response.status_code == 401:
82         print("❌ Error 401: Unauthorized - Invalid API credentials")
83         logging.error("Unauthorized Request")
84         return None
85     elif response.status_code == 500:
86         print("❌ Error 500: Server error - Payment service unavailable")
87         logging.error(f"Server Error - {response.text}")
88         return None
89     else:
90         print("❌ HTTP {response.status_code}: {response.reason}")
91         logging.error(f"HTTP Error {response.status_code}")
92         return None
93
94 except requests.exceptions.Timeout:
95     print("❌ Timeout Error: Request took too long")
96     logging.error(f"Timeout - {payload}")
97     return None
98 except requests.exceptions.ConnectionError:
99     print("❌ Connection Error: Cannot reach payment gateway")
100    logging.error(f"Connection Error - {payload}")
101    return None
102 except requests.exceptions.RequestException as e:
103    print(f"❌ Request Error: {str(e)}")
104    logging.error(f"Request Error - {str(e)}")
105    return None
106
107
108 def display_transaction_status(transaction: Dict) -> None:
109     """display transaction status."""
110     print("\n\n" + "="*55)
111     print("== TRANSACTION STATUS")
112     print("\n" + "="*55)
113     print(f"Transaction ID: {transaction.get('transaction_id', 'N/A')}")
114     print(f"Amount: {transaction.get('amount', 'N/A')} {transaction.get('currency', 'N/A')}")
115     print(f"Status: {transaction.get('status', 'N/A')}")
116     print(f"Method: {transaction.get('payment_method', 'N/A')}")
117     print(f"Timestamp: {transaction.get('timestamp', 'N/A')}")
118     print("\n\n" + "="*55)
119
120
121 def get_payment_input() -> Optional[Tuple[str, str, str]]:
122     """Get payment input from user."""
123     try:
124         amount = input("\n💰 Enter amount (or 'quit'): ").strip()
125         if amount.lower() == 'quit':
126             print("💡 Thank you for using Payment Gateway!")
127             return None
128         currency = input("\n🌐 Enter currency (USD/EUR/GBP/INR): ").strip()
129         method = input("\n💳 Enter method (credit_card/debit_card/digital_wallet): ").strip()
130         return amount, currency, method
131     except KeyboardInterrupt:
132         print("\n\n🚫 Interrupted. Exiting...")
133         return None
134
135

```

```

C:\Users\prade > OneDrive > Pradeep > python language > classnode.py > ...
136 def main():
137     """Main application loop."""
138     print("\n" + "="*53 + "\n")
139     print("\n" + "PAYMENT GATEWAY SIMULATION API".center(53) + "\n")
140     print("\n" + "(Sandbox Environment)".center(53) + "\n")
141     print("\n" + "="*53 + "\n")
142
143     while True:
144         input_data = get_payment_input()
145         if input_data is None:
146             break
147
148         amount_str, currency, method = input_data
149
150         is_valid, validated_amount = validate_amount(amount_str)
151         if not is_valid:
152             continue
153
154         if not validate_currency(currency):
155             continue
156
157         if not validate_payment_method(method):
158             continue
159
160         transaction = process_payment(validated_amount, currency.upper(), method.lower())
161
162         if transaction:
163             display_transaction_status(transaction)
164         else:
165             logging.error(f"FAILED - {validated_amount} {currency} via {method}")
166             print(f"❌ Payment failed. Details logged to {LOG_FILE}")
167
168
169 if __name__ == "__main__":
170     main()
171

```

```
Enter Stop ID (or 'quit'): C:/Users/prade/AppData/Local/Microsoft/WindowsApps/python3.12.exe "C:/Users/prade/OneDrive/Pradeep/python language/classnode.py"
Error: Invalid characters in "C:/Users/prade/AppData/Local/Microsoft/WindowsApps/python3.12.exe "C:/Users/prade/OneDrive/Pradeep/python language/classnode.py""

Enter Stop ID (or 'quit'): 101

Fetching schedule (Attempt 1/2)...
Schedule fetched successfully

=====
Stop ID: 101
=====
# Arrival Time Destination Line
=====
1 14:30 Central Bus 101
2 14:45 Central Bus 101
3 15:00 Central Bus 101
=====

Enter Stop ID (or 'quit'): 312

Fetching schedule (Attempt 1/2)...
Schedule fetched successfully

=====
Stop ID: 312
=====
# Arrival Time Destination Line
=====
1 14:25 Harbor Bus 312
2 14:40 Harbor Bus 312
3 15:10 Harbor Bus 312
=====

Enter Stop ID (or 'quit'): 205

Fetching schedule (Attempt 1/2)...
Schedule fetched successfully

=====
Stop ID: 205
=====
# Arrival Time Destination Line
=====

Enter Stop ID (or 'quit'): 101

Fetching schedule (Attempt 1/2)...
Schedule fetched successfully

=====
Stop ID: 101
=====
# Arrival Time Destination Line
=====
1 14:30 Central Bus 101
2 14:45 Central Bus 101
3 15:00 Central Bus 101
=====
```

```

Enter Stop ID (or 'quit'): 312
Fetching schedule (Attempt 1/2)...
Schedule fetched successfully

=====
Stop ID: 312
=====
#   Arrival Time   Destination   Line
-----
1   14:25          Harbor       Bus 312
2   14:40          Harbor       Bus 312
3   15:10          Harbor       Bus 312
=====

Enter Stop ID (or 'quit'): 205
Fetching schedule (Attempt 1/2)...
Schedule fetched successfully

=====
Stop ID: 205
=====
#   Arrival Time   Destination   Line
-----

=====

Enter Stop ID (or 'quit'): 312
Fetching schedule (Attempt 1/2)...
Schedule fetched successfully

=====
Stop ID: 312
=====
#   Arrival Time   Destination   Line
-----
1   14:25          Harbor       Bus 312
2   14:40          Harbor       Bus 312
3   15:10          Harbor       Bus 312
=====

Enter Stop ID (or 'quit'): 205
Fetching schedule (Attempt 1/2)...
Schedule fetched successfully

=====
Stop ID: 205
=====
#   Arrival Time   Destination   Line
-----

Stop ID: 312
=====
#   Arrival Time   Destination   Line
-----
1   14:25          Harbor       Bus 312
2   14:40          Harbor       Bus 312
3   15:10          Harbor       Bus 312
=====

```

Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots